Increasing Network Security with eBPF

Assignment #1 — State of the art

Faculty of Sciences · University of Porto

Henrique Teixeira (up202306640)
TODO (up2025XXXXX)
TODO (up2025XXXXX)

Network Security 2026/27
27 October 2026

Abstract. [TODO: 150–250 words, written last: the problem (programmable kernel hooks for network and runtime security), why it is hard (verifier as TCB, privilege boundary, new attack surface), what this report compares (hooks, production vs. research systems, threat model), and the prototype planned for Assignment #2.]

1 Introduction

[TODO: Context: eBPF runs verified programs at kernel hook points and shares state with user space through maps [1]. For network security this spans early packet filtering (XDP) to runtime enforcement (BPF LSM). State the gap: eBPF is a substrate, not a product, and it is itself security-sensitive [2].]

[TODO: Contributions as 3–4 checkable bullets, e.g. a hook comparison for two use cases, a production-vs-research comparison, a threat model for eBPF itself, and a measurable roadmap for the prototype.]

[TODO: One-paragraph outline of the report.]

2 The eBPF execution and safety model

2.1 Program lifecycle: bytecode, loading and JIT

[TODO: Restricted instruction set, `bpf()` syscall, program types, attach points, JIT compilation, pinning and lifetime [1], [3]. A small figure of the load → verify → JIT → attach pipeline helps here.]

2.2 The verifier

[TODO: What it checks: control-flow (DAG / bounded loops), register and pointer types, bounds on memory access, helper argument types, program size and complexity limits [4]. What it does *not* guarantee. Contrast with the abstract-interpretation approach of PREVAIL [5] and the critique in [6].]

2.3 Maps, ring buffers and helpers

[TODO: Map types relevant to security (hash, LPM trie, per-CPU arrays, ring buffer), helpers and kfuncs as the interface to the kernel, and how user space reads and updates policy.]

2.4 Portability: BTF and CO-RE

[TODO: Kernel-version coupling, BTF type information, CO-RE relocations with libbpf [7]. Why it matters when the same policy must run on heterogeneous kernels.]

2.5 Verifier safety vs. authorization to load

Key distinction required by the brief

The verifier answers *"is this program safe to run in the kernel?"*; the privilege model answers *"who may load and attach it?"*. A program the verifier accepts can still be malicious.

[TODO: `CAP_SYS_ADMIN` vs. `CAP_BPF`/`CAP_PERFMON`/`CAP_NET_ADMIN`, the `kernel.unprivileged_bpf_disabled` sysctl, BPF token for delegated loading, and program signing. Verify each against the current kernel docs and cite them.]

3 Hook points for security

3.1 Overview of hooks

[TODO: One short paragraph per hook: XDP [8], tc, cgroup/socket, tracepoints/kprobes, BPF LSM [9]. Choose the hook from the question, not "eBPF is faster".]

Table 1. Security-relevant eBPF hooks. [TODO: fill in and cite]

Hook	Context available	Enforcement actions	Overhead	Position
XDP	[TODO: raw frame]	[TODO: drop/ pass/redirect]	[TODO:]	driver RX
tc	[TODO: sk_buff]	[TODO:]	[TODO:]	ingress/egress
cgroup/socket	[TODO: workload, socket]	[TODO:]	[TODO:]	socket layer
kprobe/tracepoint	[TODO:]	[TODO: observe only]	[TODO:]	anywhere
BPF LSM	[TODO:]	[TODO: allow/ deny]	[TODO:]	LSM hooks

3.2 Use case A: [TODO: e.g. volumetric / DDoS filtering]

[TODO: Which hooks can solve it, what each sees, what each can enforce, and the expected overhead. Compare with nftables as the conventional baseline.]

3.3 Use case B: [TODO: e.g. process-aware connection control or telemetry]

[TODO: Same structure as use case A; the point is that a different question leads to a different hook.]

4 eBPF security systems

4.1 Production: Cilium and Tetragon

[TODO: Cilium: data-plane networking, load balancing and identity-based policy [3]. Tetragon: process and kernel observability with in-kernel enforcement [10]. Compare what state each observes, how identity is represented, and whether enforcement happens before or after the event reaches the application.]

4.2 Research: bpfbox and BPFContain

[TODO: bpfbox: per-process confinement policy enforced through LSM hooks and kprobes [11]. BPFContain: container-aware policy integrated with container managers [12]. The design trade-off between high-level semantic policy and low-level hooks; reported overhead.]

4.3 Comparison

Table 2. Production vs. research eBPF security systems. [TODO: fill in]

System	Policy level	Hooks used	Identity	Reported overhead
Cilium	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Tetragon	[TODO:]	[TODO:]	[TODO:]	[TODO:]
bpfbox	[TODO:]	[TODO:]	[TODO:]	[TODO:]
BPFContain	[TODO:]	[TODO:]	[TODO:]	[TODO:]

5 Threat model for eBPF itself

5.1 Assets, trust boundaries and assumptions

[TODO: Kernel integrity, cross-container isolation, policy integrity, secrets in kernel memory. Boundaries: user space ↔ loader ↔ verifier ↔ kernel; container ↔ host.]

5.2 Adversaries and attacks

[TODO: One paragraph each, with preconditions and impact:]

- **Malicious or compromised loader.** [TODO:]
- **Over-privileged container.** [TODO: tracing helpers reading other containers' data without a kernel exploit [2]]
- **Unsafe helper or program type.** [TODO:]
- **Verifier and JIT bugs as kernel attack surface.** [TODO: cite concrete CVEs and [6]]
- **Policy bypass.** [TODO: TOCTOU, semantic gaps between hook and intent, event loss under load]

5.3 Mitigations

[TODO: Capability restriction, disabling unprivileged BPF, seccomp/AppArmor profiles for containers, BPF token, signing/auditing of programs, and which of these the prototype will apply.]

6 Limitations and open challenges

[TODO: The verifier as part of the TCB; observability vs. enforcement (false positives, availability impact); portability across kernels; event loss; operational complexity. Compare approaches rather than list them.]

7 Proposed prototype and evaluation criteria

7.1 Scope

[TODO: Pick one, distinct from the other group on this topic: XDP filtering with counters, connection telemetry correlated with processes, or a BPF-LSM policy protecting one resource. State the security property it demonstrates.]

7.2 Metrics

Table 3. Evaluation criteria for Assignment #2.

Metric	How it is measured	Target / baseline
Correctness	[TODO: positive and negative test cases]	[TODO:]
Throughput / event rate	[TODO:]	[TODO: vs. nftables]
Latency	[TODO:]	[TODO:]
CPU overhead	[TODO:]	[TODO:]
Memory / map usage	[TODO:]	[TODO:]
Event loss / false positives	[TODO:]	[TODO:]
Privileges required	[TODO: capabilities before and after]	[TODO:]

7.3 Environment

[TODO: Isolated VM(s), kernel version, clang/libbpf versions, traffic generator, and whether the prototype runs natively or in a fallback mode.]

8 Implementation roadmap for Assignment #2

Table 4. Milestones, dependencies and validation. [TODO: set dates once the Assignment #2 deadline is on Moodle]

#	Milestone and deliverable	Depends on	Validation	Week
M1	Minimal program loads and attaches	—	[TODO:]	[TODO:]
M2	User-space controller / telemetry	M1	[TODO:]	[TODO:]
M3	Policy and test cases	M2	[TODO:]	[TODO:]
M4	Benchmark harness and baseline	M3	[TODO:]	[TODO:]
M5	Threat-model validation (privilege audit)	M3	[TODO:]	[TODO:]
M6	Reproducibility artefacts and README	M4, M5	[TODO:]	[TODO:]

9 Ethical and legal considerations

[TODO: All experiments in isolated, authorized infrastructure; no public Internet targets; offensive tests (e.g. cross-container reads) only against our own containers.]

10 Conclusion

[TODO: What the comparison showed, the main open challenges, and how the prototype will address the research questions.]

Bibliography

[1] The Linux Kernel Documentation, “BPF Documentation.” [Online]. Available: https://docs.kernel.org/bpf/

[2] Y. He *et al.*, "Cross Container Attacks: The Bewildered eBPF on Clouds," in *32nd USENIX Security Symposium (USENIX Security 23)*, USENIX Association, 2023. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/he

[3] Cilium Authors, "BPF and XDP Reference Guide." [Online]. Available: https://docs.cilium.io/en/stable/reference-guides/bpf/

[4] The Linux Kernel Documentation, "eBPF Verifier." [Online]. Available: https://docs.kernel.org/bpf/verifier.html

[5] E. Gershuni *et al.*, "Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '19)*, ACM, 2019, pp. 1069–1084. doi: 10.1145/3314221.3314590.

[6] J. Jia, R. Sahu, A. Oswald, D. Williams, M. V. Le, and T. Xu, "Kernel Extension Verification is Untenable," in *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS '23)*, ACM, 2023, pp. 150–157. doi: 10.1145/3593856.3595892.

[7] libbpf Authors, "libbpf-bootstrap: Scaffolding for BPF Application Development with libbpf and BPF CO-RE." [Online]. Available: https://github.com/libbpf/libbpf-bootstrap

[8] T. Høiland-Jørgensen *et al.*, "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel," in *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '18)*, ACM, 2018, pp. 54–66. doi: 10.1145/3281411.3281443.

[9] The Linux Kernel Documentation, "LSM BPF Programs." [Online]. Available: https://docs.kernel.org/bpf/prog_lsm.html

[10] Cilium Authors, "Tetragon: eBPF-based Security Observability and Runtime Enforcement." [Online]. Available: https://tetragon.io/docs/

[11] W. Findlay, A. Somayaji, and D. Barrera, "bpfbox: Simple Precise Process Confinement with eBPF," in *Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop (CCSW '20)*, ACM, 2020. doi: 10.1145/3411495.3421358.

[12] W. Findlay, D. Barrera, and A. Somayaji, "BPFContain: Fixing the Soft Underbelly of Container Security." [Online]. Available: https://arxiv.org/abs/2102.06972